

Take-Home Assignment: Service Request Router Agent

Time: One focused working day. If you're running long, cut scope — a small system that works end to end beats a large half-built one.

The problem

An operations desk at a field-services company (HVAC, plumbing, electrical, general repairs) receives unstructured customer requests by portal, email, or phone note. A dispatcher turns each into a job: pick the right **Service Workflow template** from a fixed catalogue, set the urgency, and make sure the crew has the intake info it needs.

Build an **agent that does the first pass**. What interests us most is the **agentic loop**: how your system reads a request, works out what it knows and what it's missing, and acts — decide, ask, or escalate. (How the loop would *keep improving* in production is a question we ask you to think through in the write-up, not something to build.) Given one raw request, it should:

1. **Extract** a structured picture: what, where, real urgency, equipment/area, constraints, safety signals.
2. **Detect gaps and conflicts** — missing intake information, and requests that contradict themselves. Stated urgency and real urgency are not always the same.
3. **Clarify and close the loop** — when the decision depends on missing or conflicting info, ask a few targeted questions, take the answers back in (simulated answers are fine), and produce a final decision.
4. **Select** the best-fitting template — or decide none fits.
5. **Attach confidence and routing** — at minimum: confident recommendation / needs clarification / route to a human. The banding is your design.
6. **Explain** briefly: which template, why, what's missing, what would change the call.

The catalogue is deliberately small and imperfect: near-duplicate templates (e.g. *HVAC Emergency* vs *HVAC Routine Maintenance*) and gaps where some requests match nothing well. You decide the architecture — no required framework, agent count, or orchestration style.

Starter materials

- **service_catalogue.json** — the fixed template catalogue (id, category, urgency tier P1/P2/P3, required intake fields, typical signals, SLA, prerequisites). Treat it as data that could change.
- **test_requests.json** — eight seed requests of uneven quality. No answer key. **Add at least three genuinely hard cases of your own** and say why each is hard.

Requirements

- [] At least one **real LLM API call** (not mocked); credentials via environment variables. Any major provider is fine — we'll run it with our own Anthropic or OpenAI key, so prefer one of those or include clear instructions.
- [] **Structured output validated against a schema**, with deliberate behaviour on off-shape responses and API errors (repair / retry / escalate — not a crash).
- [] An explicit **confidence / escalation mechanism**.
- [] The **clarification loop closes**: answers go back in → final decision.
- [] A **measured evaluation** over the seeds + your added cases. At minimum, measure how often the agent makes the right *decide-vs-escalate* call; add any other metrics you find useful.
- [] **One real iteration**: a first version that measurably underperforms, a deliberate change, and a re-run that moves the number.

Keep it simple — use a framework, vector store, or multiple agents only if the problem needs it. Shape the agent's behaviour however works: instructions, few-shot examples, rules, prompt iteration, error analysis. Fine-tuning is not expected.

Deliverables

1. **Runnable code** + a short README: setup, how to run, required env vars, example input, known limitations.
2. **One written explanation, max 3 pages** (raw outputs and attachments don't count against the cap), covering: - Architecture (a diagram or 2-3 sentences) and the end-to-end flow. - How you shaped and evaluated the agent — the number(s) you measured, what failed in the first version, what you changed, and how the number moved. Attach the raw before/after outputs. - Your most important decision, hardest trade-off, the **two biggest production risks**, and what you deliberately did *not* build. - **The agentic loop, looking forward**: in production, what would run on its own, what feedback would flow back in (e.g. dispatcher corrections), and what you would let the agent do autonomously over time. You don't need to build this — describe how you envision it. - Would **MCP** (Model Context Protocol) add meaningful value here? Where — or why not? - **Working with AI**: where you relied on AI tools vs. did it yourself, and one concrete moment you overrode or corrected an AI suggestion.
3. **One proof-it-ran artifact** — an eval summary, log, or screenshot. (Optional screen recording, max 5 minutes.)

Scope & submission

No UI, no deployment, no production polish. Document assumptions instead of resolving every ambiguity. Use AI tools as you normally would, but you must understand and be able to defend everything you submit; we will ask. Submit a repo link or zip — and if anything blocks you, just ask.

Appendix — starter data at a glance

The JSON files are the source of truth; this is orientation only.

Urgency tiers: **P1** = safety risk / active damage, same-day. **P2** = down/degraded, no immediate safety risk, 24–48h. **P3** = routine/cosmetic/preventative, 5–10 business days.

id	name	category	urgency
HVAC_EMERGENCY	HVAC Emergency Repair	HVAC	P1
HVAC_MAINT	HVAC Routine Maintenance / Servicing	HVAC	P3
PLUMB_LEAK	Plumbing Leak / Water Ingress Response	Plumbing	P1
PLUMB_STD	Standard Plumbing Repair	Plumbing	P3
ELEC_FAULT	Electrical Fault / Power Loss Response	Electrical	P1
ELEC_INSTALL	Electrical Installation / Fit-out	Electrical	P3
LOCK_ACCESS	Lock, Door & Access Response	Security / Access	P2
HANDYMAN_MINOR	General Handyman / Minor Repairs	General	P3

The 8 seed requests in `test_requests.json` range from clear to deliberately ambiguous.